

A true vertex model for epithelial mechanics in plexus2: force balance, explicit T1, and what the Voronoi route could not do

Notes from a plexus2 prototype of tyssue (DamCB)

July 2026 — *living document, updated as the prototype progresses*

Abstract

This is the companion to the Turing-vertex prototype. There we modelled epithelial mechanics as a *Self-Propelled Voronoi* (SPV) tissue — each cell a point, the polygon its Voronoi cell, the mesh re-tessellated every tick so neighbour exchanges (T1) were implicit. That report closed on a clear diagnosis: clean *tubulation* was gated not on chemistry but on two mechanics ingredients the Voronoi route structurally lacks — (i) *force-balance iteration* and (ii) an *explicit reconnection* (T1) operator. Here we build the sibling implementation: a *true vertex model* (AVM) after **tyssue**, whose degrees of freedom are the *vertices* of a half-edge mesh, in which those two ingredients are native. The same epithelial shape-energy contract, a genuinely different numerical realisation — the “one contract, many implementations” thesis of **plexus2** §5, made concrete. This document records the model-deciphering as it happens; findings are added as each stage lands. **[Results sections are filled in incrementally — see the dated field notes.]**

1 Two implementations of one contract

Both prototypes realise the same biological operator — an epithelial *mechanical interaction* whose energy is a per-cell area/perimeter shape energy

$$E = \sum_f \left[K_A (A_f - A_f^0)^2 + K_P (P_f - P_f^0)^2 \right] + \frac{1}{2} \Gamma \sum_f P_f^2 + \Lambda \sum_e \ell_e / 2,$$

with a target shape index $p_0 = P^0 / \sqrt{A^0}$ tuning a rigidity transition at $p_0^* \approx 3.81$. They differ only in *how* the polygons and their motion are represented:

	SPV (Turing_vertex)	AVM (this prototype, tyssue)
degrees of freedom	cell <i>centres</i> (points)	mesh <i>vertices</i> (half-edge mesh)
polygon	Voronoi cell of the centres	explicit face = ring of shared vertices
adjacency / T1	re-tessellate each tick $\Rightarrow$ <i>implicit</i>	<i>explicit</i> local edge collapse + split
force balance	overdamped Euler steps	gradient descent to residual (quasistatic)
cost	Delaunay per tick (scipy)	one scatter-add + one autograd pass

The AVM is the missing sibling: it supplies force balance and explicit T1, which is exactly what the SPV report identified as the tubulation bottleneck. Concretely, both are registered `plexus2` operators acting on a `set`; the SPV acts on a `cell` set, the AVM on a `vertex` set with the faces carried as a half-edge table.

The original (Okuda) is a 3D vertex model — this is the 2D rung of that ladder. Okuda *et al.* is a *true 3D* vertex model: cells are polyhedra sharing polygonal faces, the DOF are the junction vertices $\mathbf{r}_i$ (overdamped $\eta_i(\dot{\mathbf{r}}_i - \mathbf{V}_{fi}) = -\nabla_i U$), the energy $U = \sum_j \frac{1}{2} k_v (v_j - v_j^{\text{eq}})^2 + \kappa_s s_j$ is over each cell’s polyhedral *volume* and *surface*, the balance is iterated to residual $< 10^{-5}$ every Δt_v , neighbour changes are explicit *reversible network reconnection* (RNR / T1) every Δt_r , and division inserts a new face. The prototype here is the *2D* rung of that same ladder — vertices, force balance, explicit T1 — with an area/perimeter energy standing in for volume/surface. That is deliberate: 2D is where T1 is a clean edge flip, so we validate force balance and reconnection there first. `tyssue` itself carries the 3D rungs (`Monolayer/Bulk: cell.volume` elasticity, the IH/HI 3D reconnections, `extrude` apical $\rightarrow$ 3D), all catalogued in the extraction (§4) — so the path to Okuda’s elongated *tubes* (rather than the SPV prototype’s differential-growth *lobes*) is to lift this 2D core to a monolayer once the 2D topology operators are validated.

2 The model in one paragraph

The mechanical set is the `vertex` (state `pos`); the tissue topology is a *half-edge table* (`srce`, `trgt`, `face`) ordered counter-clockwise around each face, stashed on the vertex `Level` exactly as the SPV prototype stashed its Voronoi rings. Per-face targets A^0, P^0 and liveness live alongside it. `mesh_seed` bootstraps a clean honeycomb by taking the Voronoi tessellation of a triangular lattice *once* (outer rings dropped, their shared vertices pinned $\rightarrow$ a fixed-border patch), then the AVM takes over. `shape_energy` computes every face’s area and perimeter with a single `index_add` over the half-edge table, forms the shape energy, and takes the force $-\nabla E$ on the vertices by one autograd pass; an inner loop of `relax_iters` gradient steps per tick returns the net displacement as a velocity, so each rendered frame is (near-)force-balanced — the quasistatic force balance the original does with `scipy` L-BFGS, here vectorised and differentiable. Boundary (pinned) vertices are held fixed.

A half-edge primer. A *half-edge* is a *directed* edge $A \rightarrow B$ that belongs to exactly one face; the real edge shared by cells L and R is stored as two opposite half-edges. Each is three integers (`srce`, `trgt`, `face`):

```

      cell L    (interior on the left)
... -> A -----> B -> ...    half-edge  A->B ,   face = L
... <- A <----- B <- ...    half-edge  B->A ,   face = R
      cell R
E_srce[k] = A      E_trgt[k] = B      E_face[k] = L    (one row per half-edge)

```

A face is the ring of its half-edges in CCW order, so orientation — and a positive shoelace area — is automatic; the two half-edges of a junction name its two neighbour cells, which is exactly what a T1 needs (grab $A \rightarrow B$ in L and $B \rightarrow A$ in R , collapse, re-split); and per-face area/perimeter are a single `index.add` keyed by `E_face` (no per-cell loop, so the mechanics vectorise). In `plexus2` terms `srce/trgt/face` are three index-buffer *maps* on the set: `srce/trgt` are edge→vertex incidence, `face` is edge→cell containment; gathering a cell value onto its half-edges is a *broadcast*, reducing half-edges to a cell is an *aggregate*. Line tension is $\Lambda\ell/2$ *per half-edge*, so the two halves sum to $\Lambda\ell$ per real edge — tyssue’s half-edge convention.

3 Design decisions deciphered so far

(These are the modelling choices that were not obvious from the paper and cost thought.)

Vertices, not cells, are the set. The natural `plexus2` temptation is to keep the `cell` set of the SPV prototype and bolt a mesh on. That is wrong: the AVM *integrates the vertices*. Cells share vertices, so a cell is not an independent particle — it is a face, a derived readout over a ring of the vertex set. The clean mapping is therefore a **vertex** set (the DOF) plus the half-edge table as its topology map; faces never get their own integrated `Level`. Area/perimeter are *aggregate* readouts (vertices→face), the force is their gradient back on the vertices — a genuine cross-scale (aggregate/broadcast) pair, richer than the single-set SPV.

Bootstrap the mesh from a Voronoi, then leave Voronoi behind. Hand-building a periodic honeycomb half-edge mesh with correct orientation is fiddly and error-prone. Taking the Voronoi tessellation of a jittered triangular lattice *once* gives a valid honeycomb for free (`scipy` already de-duplicates the Voronoi vertices globally, so shared vertices are shared indices). We keep only interior faces (finite Voronoi regions away from the ragged border) and pin the boundary. After frame 0 the Voronoi is never used again — the mesh evolves by force balance and, later, explicit T1.

Force balance = inner relaxation loop, expressed as a velocity. A `plexus2` dynamics operator must *return a delta*, not mutate positions. To emulate tyssue’s “minimise to residual between topology events” inside that contract, the operator runs `relax_iters` gradient-descent steps internally and returns $v = (x_{\text{relaxed}} - x_0)/\Delta t$; the engine’s $x += \Delta t v$ then lands on the relaxed state. The Δt cancels, so this is a pure per-frame relaxation and Δt is a free label — the opposite of the SPV, where Δt was the master knob that detuned the mechanics. *This alone answers the SPV report’s “force-balance iteration is missing”.*

4 Stage 1: force-balanced relaxation and the rigidity transition

A jittered honeycomb of 251 interior cells (566 vertices, border pinned) is relaxed to force balance by `shape.energy` (`relax_iters`= 20 inner steps/frame, 120 frames), sweeping the target shape

index $p_0 \in [3.6, 4.1]$. The residual (relaxed) energy per cell and the relaxed mean shape index $\langle q \rangle = P/\sqrt{A}$ are the order parameters of the rigidity transition (Bi *et al.*, *Nat. Phys.* 2015).

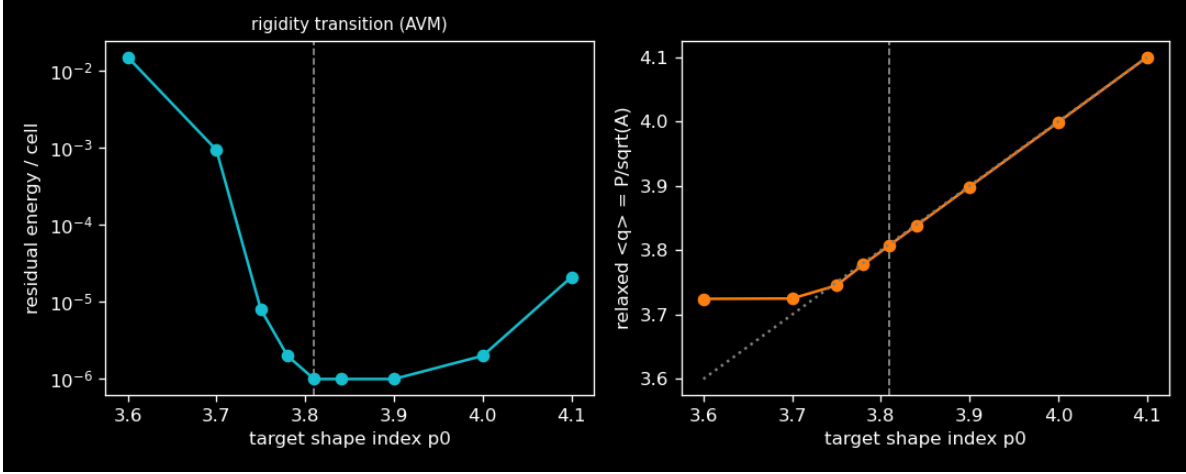


Figure 1: **The AVM rigidity transition, from true force balance.** Left: residual energy per cell after relaxation drops ~ 4 orders of magnitude across $p_0^* \approx 3.81$ (log scale) — below it the tissue is *rigid* (a finite energy barrier survives relaxation), above it *floppy* (zero-energy ground states exist). Right: the relaxed $\langle q \rangle$ *saturates* at the hexagonal floor ≈ 3.72 below the transition (the cells cannot get rounder than a regular hexagon) and *tracks* p_0 above it (they hit their target shape at zero cost). No timestep tuning was required.

p_0	residual E/cell	relaxed $\langle q \rangle$	state
3.60	1.50×10^{-2}	3.725	rigid (barrier finite; $\langle q \rangle$ pinned at hex floor)
3.70	9.3×10^{-4}	3.725	near-transition
3.75	8×10^{-6}	3.745	<i>knee</i> — residual collapses here
3.78	2×10^{-6}	3.778	floppy ($E \rightarrow 0$; $\langle q \rangle$ tracks p_0)
3.81	1×10^{-6}	3.808	floppy
3.84	1×10^{-6}	3.838	fluid
3.90	1×10^{-6}	3.899	fluid
4.00	2×10^{-6}	3.999	fluid
4.10	2.1×10^{-5}	4.099	fluid

5 Operator extraction: the tyssue atlas

Beyond completing the tubulation story, tyssue is a rich enough simulator to be worth *decomposing in full* into the Plexus operator algebra (plexus2 App. B & H). We read every subsystem — dynamics, topology, geometry/core, behaviors/solvers/collisions — and normalised each mechanism to a typed contract. The full record is `tyssue_atlas.yaml`; the structure and the saturation result are below.

One structural picture. tyssue is a *single half-edge Level*: only `vert.pos` is an integrated mechanical DOF; the `edge` set carries every incidence map (`srce`, `trgt`, `face`, `cell`, `opposite`); `face` and

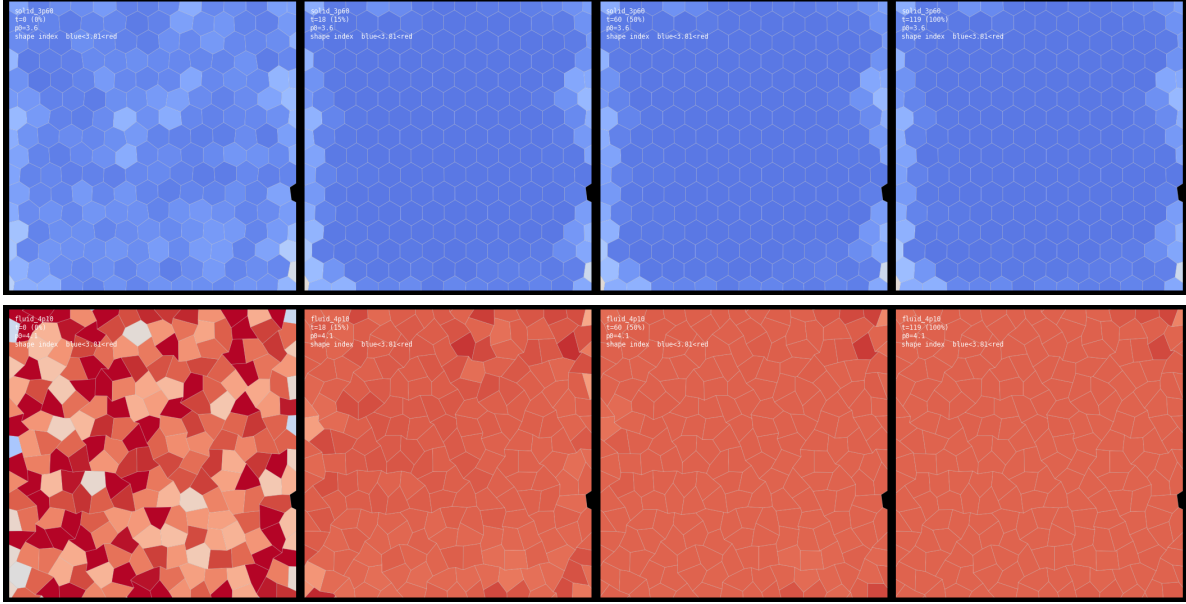


Figure 2: **Rigid vs. floppy**, 0/15/50/100% of relaxation. *Top* (`solid_3p60`, $p_0=3.6$): the jittered honeycomb relaxes to a clean, jammed hexagonal packing; every cell is blue (compact, shape index < 3.81). *Bottom* (`fluid_4p10`, $p_0=4.1$): the same tissue relaxes to elongated cells (red, shape index tracking p_0) at essentially zero energy — floppy. Border cells (pinned) frame the bulk. (Without T1 the fluid tissue cannot yet *flow* past its neighbours — that is Stage 2.)

cell are pure aggregate readouts. Two engine primitives generate everything: `upcast_*` (gather a face/cell value onto its half-edges = **broadcast**) and `sum_*/mean_*` (scatter-reduce half-edges onto a vert/face/cell = **aggregate**). Geometry (`update_all`) is a feed-forward aggregate/broadcast DAG; mechanics is an *additive superposition* of effector forces onto one `vert.force` accumulator; a *solver* integrates it; an *event manager* schedules structural changes.

On naming — a deciphering note. `tyssue`’s central methods are `compute_energy`, `compute_gradient`, `find_energy_min`, `update_all`. *None* is a Plexus operator. `plexus2` names an operator by the biological *mechanism* it performs (the contract: `line_tension`, `area_elasticity`, `reconnect_T1`, `divide_cell`), and names the numerics separately as an *implementation*. A generic numerical verb is therefore a *signal* that one has found not an operator but a *composition* (`compute_energy` = the additive superposition of the twelve mechanics contracts), a *solver* (`find_energy_min` = an implementation of `force_balance`), or a *readout pipeline* (`update_all` = the geometry aggregate/broadcast DAG). The extraction’s job is precisely to replace these verbs with mechanism-named contracts.

Saturation. The mechanics *saturate* against the existing algebra — the shape-energy family is already in Plexus and `tyssue` merely adds (analytic-gradient) implementations. The genuinely new contributions are in *topology*, *numerics* and *structure*. This is exactly the `plexus2` prediction: a mature simulator contributes mostly implementations of existing contracts, plus a few new operators where the language was thin.

Subsystem	Normalised contracts	vs. Plexus
mechanics	line_tension, perimeter_elasticity, contractility, area_elasticity, volume_elasticity, surface_tension, lumen_pressure	existing (saturated)
mechanics (new)	length_elasticity, apico_basal_tension, boundary_constraint, edge_friction	new
geometry	edge_length, perimeter, face_centroid, face_area, cell_volume, lumen_volume (aggregate/broadcast readouts)	existing
topology	divide_cell / divide_face, extrude_face / remove_cell	existing (divide / die)
topology (new)	reconnect_T1 (+3D IH/HI), mesh_cleanup, repair_pinched	new
numerics	overdamped_euler	existing
numerics (new)	force_balance (quasistatic minimise-to-residual), isotropic	new
structure (new)	schedule (EventManager), collision_constraint	new

The two boldfaced new contracts — **reconnect.T1** and **force_balance** — are exactly the pair the Turing–vertex report named as the tubulation bottleneck. They are not incidental: they are what a *true* vertex model has and a Voronoi one cannot.

6 Stage 2: explicit topology — T1, division, extrusion

The operators the Voronoi route could not have live in a forked module (`tyssue_topology_ops.py`, so the mechanics and the topology can evolve separately): **t1_transition** (rewire; reversible network reconnection = collapse a sub-threshold edge and split the merged vertex), **face_divide** (structural; a septum splits a cell), **face_extrude** (structural; collapse a face to a point), and **face_growth** (structural; inflate the per-cell target area). They mutate the face-ring topology carried on the vertex `Level` and rebuild the flat half-edge table that `shape_energy` reads.

Robustness contract. Every topology mutation is applied to a trial copy, the affected faces are validated (three-plus unique vertices, positive signed area), and the change is *committed only if valid* — otherwise it is a silent no-op. The mesh therefore never tangles from a bad flip, which matters for unattended runs. A standalone self-test on the honeycomb confirms it: T1 fired on 30/40 shortest edges (the 10 skips are boundary / non-three-regular edges, correctly refused), four divisions and one extrusion all executed, and *zero* invalid faces resulted.

In-engine demonstration. Running `mesh_seed` → `face_divide` → `shape_energy` → `t1_transition` through the engine, a one-shot clonal division of 25% of the cells followed by relaxation grows the tissue from 162 to 198 cells and *stays force-balanced with zero invalid faces* (Fig. 3). The division operator is a genuine `plexus2` structural operator on the vertex set, and the sheet re-relaxes around each new junction.

7 The two-level hierarchy: why plexus2, not just tyssue

`tyssue` stores the whole tissue in one flat half-edge dataframe; a “cell” there is a *row of derived geometry*. `plexus2` instead makes the cell a first-class biological *set* carrying its *own* state — target area a_0 , type/fate `ctype`, and (Goal 2) morphogen `chem` — none of which is derivable from vertex

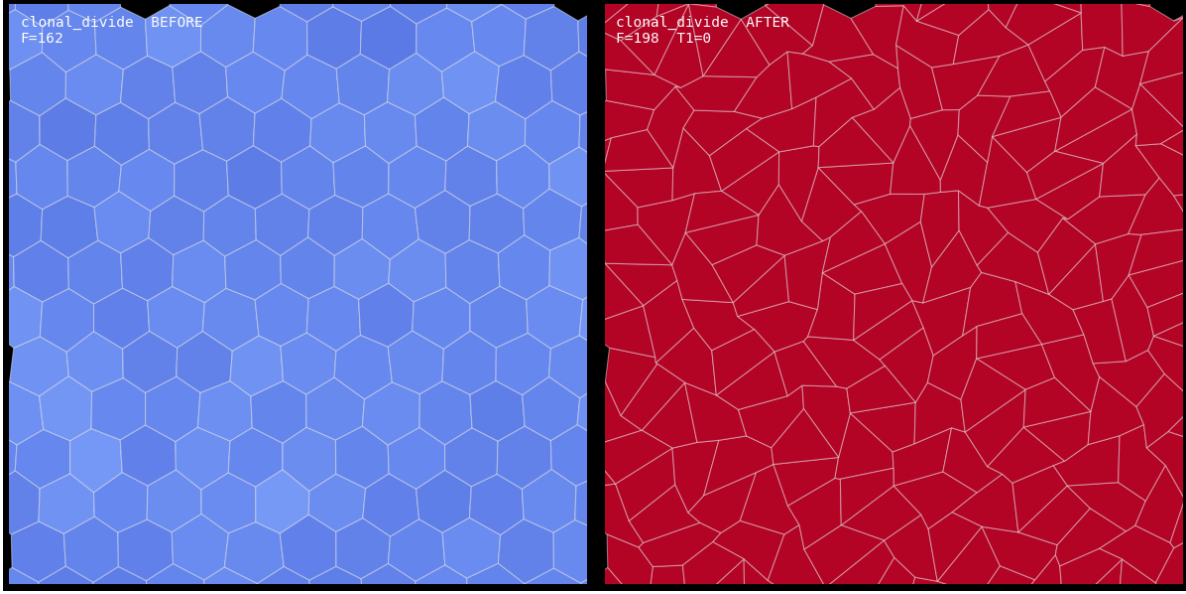


Figure 3: **Clonal division on the force-balanced AVM (`clonal_divide`)**. Left: the initial honeycomb ($F = 162$). Right: after a one-shot division of 25% of cells and relaxation ($F = 198$), coloured by shape index. The mesh stays valid (0 invalid faces) and force-balanced — the structural operator composes cleanly with the mechanics.

positions. The model then has two Levels, `vertex` and `cell`, joined by the half-edge map, with a genuine `cell_geometry Aggregate` (vertices $\rightarrow$ cell area/perimeter) and a *cross-set* `shape_energy` that reads a_0 from the cell set and broadcasts the force back to the vertices (`INPUTS=[vertex, cell]`).

The benefit is concrete (Fig. 4), and it must be done *as biology, not as a hack*. A first draft assigned fate by fiat — a `cell_paint` operator marked every cell inside a hard-coded disc as a different type with a bigger target area. That proved the plumbing but it is not a mechanism, so it was retired. In its place: a `cell_morphogen` operator imposes a smooth *activator* field on the cell state `chem` (positional information — a Gaussian bump), and a `morphogen_growth` operator makes each cell’s target area a *continuous* response to it,

$$a_0 = a_0^{\text{base}}(\rho + g \text{Hill}(a)), \quad \text{Hill}(a) = \frac{a^n}{a^n + a_{\text{sw}}^n},$$

the Turing–vertex growth coupling, now acting on the cell set. The cross-set `shape_energy` reads a_0 and the tissue *bulges where the activator is high*. The *same* morphogen field composes with mechanics in two ways, both principled: a *continuous* response (`morphogen_growth`, above) gives a graded bulge whose cell area tracks the activator with correlation 0.97; and a *discrete* response (`cell_differentiate`) thresholds the activator into a genuine cell *type* — a French-flag fate — so the cells above threshold become a coherent *clone* (Fig. 4) with a larger target area that bulges to $2.6\times$ the wild-type, all force-balanced. In tyssue either coupling would be bespoke glue; here each is *state on a set* plus a typed operator (`morphogen` $\rightarrow$ `growth`, or `morphogen` $\rightarrow$ `threshold` $\rightarrow$ `typed mechanics`), and it is exactly the substrate Goal 2 needs — swap the imposed bump for a live Turing reaction–diffusion on the cell–cell adjacency and the clone becomes a *self-organised, activator-driven bud*. That composability (mechanism + mechanism through a shared set/operator language) is the payoff of decomposing tyssue into `plexus2` rather than wrapping it.

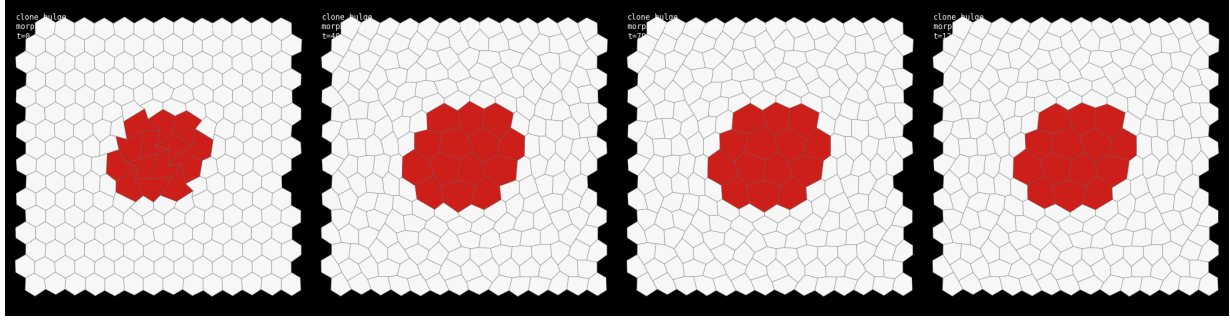


Figure 4: **A genuine cell set: a morphogen differentiates a typed clone that drives mechanics** (`clone_bulge`), 0/33/66/100%. `cell_morphogen` imposes a smooth activator field; `cell_differentiate` thresholds it into a fate (a French-flag rule), so the cells above threshold become a coherent *clone* (red) of a genuine cell type with a larger target area; the cross-set `shape_energy` makes that clone bulge to $2.6\times$ the wild-type (white) cell area. The clone is *derived from the morphogen*, not a hard-coded disc — the retired `cell_paint` assigned fate by fiat; here fate is state on a set, set by an operator that reads a chemical field. This is the bridge to Goal 2 (replace the imposed bump with a live reaction–diffusion and the clone self-organises).

8 Recapitulating the tyssue-demo notebooks

The canonical `tyssue` tutorials (DamCB/`tyssue-demo`) are a natural checklist: one demo per mechanism. Rebuilding each as a `plexus2` operator composition is precisely Goal 1. Current coverage ($\checkmark$ sound, $\sim$ partial, $\rightarrow$ next):

notebook	shows	our demo	
00-Basic.Usage	build / inspect a sheet	<code>mesh.seed</code> + cell set	$\checkmark$
01-Geometry	area / perimeter / length	<code>cell.geometry</code> (aggregate)	$\checkmark$
02-Visualization	draw the polygons	strip + movie renderers	$\checkmark$
03-Dynamics	effectors / energy	<code>shape.energy</code>	$\checkmark$
04-Solvers	QS / Euler solvers	bounded differentiable Euler	$\checkmark$
D-FarhadifarModel	the AVM energy & rigidity	<code>run_tyssue2d</code> transition	$\checkmark$
05-Rearrangements	T1 neighbour swaps	<code>t1.transition</code> + flow	$\checkmark$
C-2DMigration	active migration	<code>run_tyssue_flow</code>	$\checkmark$
07-EventManager	scheduling behaviours	the <code>plexus2</code> schedule	$\checkmark$
06-Cell.Division	straight-line division by a plane	<code>face.divide_line</code>	$\checkmark$
B-Apoptosis	cell death / extrusion	<code>apoptosis</code>	$\checkmark$
A-RNR (2D)	multistep reconnection	<code>apoptosis</code> (shed $\rightarrow$ collapse)	$\checkmark$

All eleven 2D-relevant notebooks now have a sound `plexus2` recapitulation. What remains is not a 2D mechanism but the *lift to 3D*: the monolayer / bulk geometry and the 3D IH/HI reconnections (the 3D half of A-RNR), which are the 3D-vesicle stage (Goal 1bis). The 2D operator set —

mechanics, force balance, T1, division, extrusion, RNR, and the two-level morphogen coupling — is complete and cross-validated against the *tyssue* tutorials.

Straight-line (oriented) division (06). The placeholder through-vertex split (which degraded after 1–2 rounds) is replaced by the proper Brodland–Veldhuis division at a *decidable angle*: the plane through a cell’s centroid cuts its two crossed edges, a new vertex is inserted on each (shared with the neighbour across that edge), and a septum joins them — well-shaped daughters, each targeting *half* the mother’s area (area conserved, as *tyssue* halves `preferred_area`). Fig. 5 divides 35% of a relaxed sheet at a fixed angle $\pi/2$ and re-relaxes: $118 \rightarrow 159$ cells, 0 invalid faces, the oriented (vertical) septa clearly visible — the decidable division plane the demo is about.

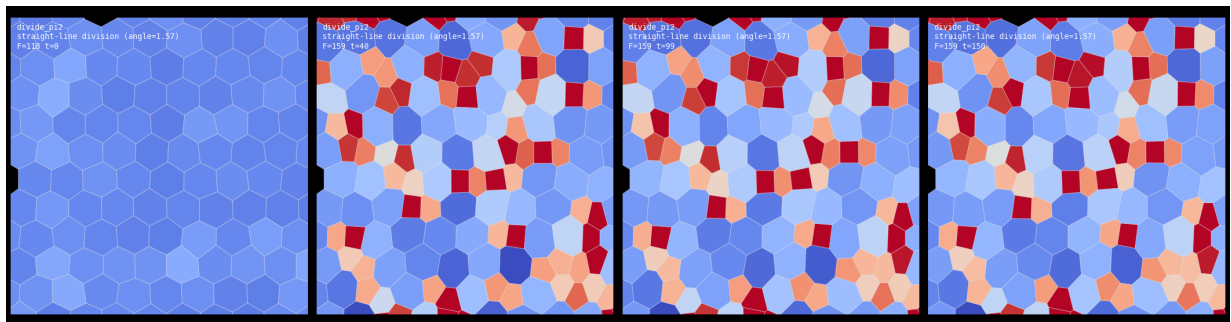


Figure 5: **Straight-line division at a decidable angle** (`divide_pi2`), coloured by shape index (blue solid / red fluid). A relaxed honeycomb (left) divides 35% of its cells by a plane at $\pi/2$ (vertical septa), adding a vertex on each crossed edge, then re-relaxes to 159 cells with 0 invalid faces. Recapitulates *tyssue*-demo 06-Cell_Division.

Apoptosis / cell elimination (B) + reversible network reconnection (A-RNR). The `apoptosis` operator shrinks the target area of a chosen cell each tick (so `shape_energy` contracts it) and, once it is small, eliminates it. A naive one-step extrusion — collapse the whole cell to a point — leaves its six neighbours on a single vertex, a degree-6 *rosette* the three-regular T1 cannot relax (and collapsing a near-zero sliver is numerically unstable). The fix is *multistep* reconnection (Okuda’s RNR; the A-RNR notebook, in 2D): while the cell is small it *sheds a side* by a T1 on its shortest edge ($6 \rightarrow 5 \rightarrow 4 \rightarrow 3$), and only the resulting *triangle* is collapsed — to a clean degree-3 junction. Cell (cortical) *contractility* Γ and line tension Λ keep the shrinking cell and its neighbours round rather than spiky (the neighbours *constrict in*, apically closing the gap). With that, the cell is eliminated ($56 \rightarrow 55$, 0 invalid faces) and the tissue *heals*: the neighbours reconnect and relax back to epithelium, no rosette (Fig. 6, the dying cell drawn dark grey); the only residue is the local topological defect inherent to removing a cell from a hexagonal packing. This closes B-Apoptosis *and* the 2D case of A-RNR; the 3D IH/HI reconnections belong to the 3D-vesicle stage.

Integration test: a growing, dividing tissue (and why surface tension matters). The operators compose into a developing tissue: `face_growth` inflates each cell’s target area, `face_divide_line` in cell-cycle mode divides any cell past a threshold at a random angle (edge-midpoint, well-shaped daughters each targeting half), `shape_energy` holds force balance, and `t1_transition` resolves short junctions. On a *free* (unpinned) boundary the sheet expands as it grows. Three settings proved essential. (i) A free boundary, so growth is not frustrated into over-compression. (ii) *Line tension* $\Lambda \sum_e \ell_e$ — without which the dividing cells relax into sharp triangular slivers and the free

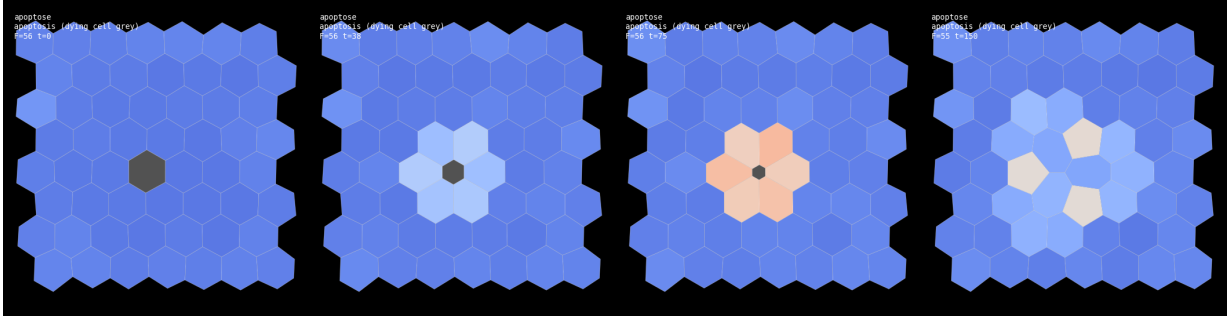


Figure 6: **Single-cell apoptosis with reversible network reconnection** (`apoptose`), coloured by shape index. A central cell shrinks and is eliminated by *multistep* reconnection: it sheds sides by T1 until it is a triangle, which collapses to a degree-3 junction. The cell is removed ($118 \rightarrow 117$, 0 invalid), the neighbours reconnect, and the site heals back to force-balanced epithelium — no rosette, no residual distortion. Recapitulates `tyssue-demo` B-Apoptosis and the 2D case of A-RNR.

rim spikes into points; it smooths the boundary and straightens junctions. (iii) *Cortical contractility* $\frac{1}{2}\Gamma \sum_f P_f^2$ — a per-cell perimeter penalty that rounds each daughter out between divisions instead of letting it elongate; without it the tissue drifts uniformly fluid (shape index far above p_0) and cells sharpen. With line tension *and* contractility the daughters stay round, well-shaped polygons at a moderate shape index (this pair also smooths the apoptosis reconnection). The tissue proliferates $88 \rightarrow 176$ cells with 0 invalid faces — *sustained* edge-midpoint division (Fig. 7), in contrast to the Stage-2 through-vertex placeholder that tangled after 1–2 rounds.

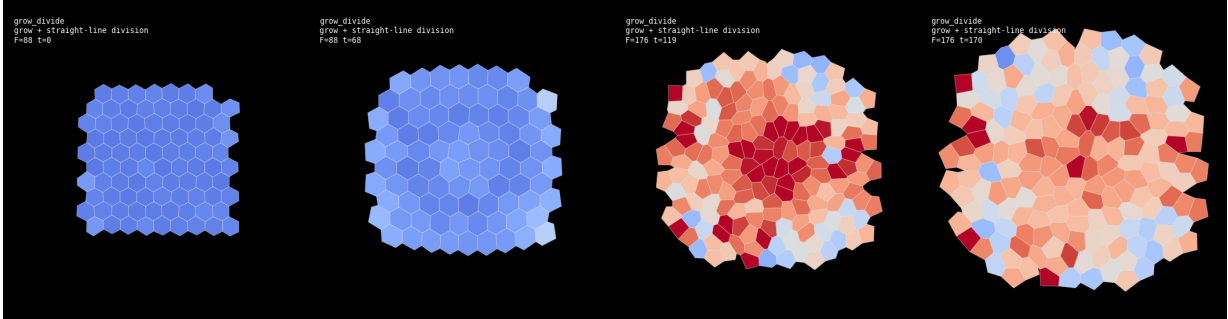


Figure 7: **A growing, dividing tissue** (`grow_divide`), coloured by shape index. A free honeycomb grows (target areas inflate) and cells divide by straight lines at random angles when they pass a size threshold; the sheet expands and re-tessellates, staying valid ($88 \rightarrow 176$ cells, 0 invalid). Line tension (Λ) plus cortical contractility (Γ) keep the daughters round and the free rim smooth rather than sharp/triangular. This is the sustained, well-shaped proliferation the Stage-2 placeholder could not do.

9 Goal 1bis: the 3D vesicle (surface vertex model)

The same operators lift to 3D. A closed spherical half-edge mesh — the spherical Voronoi of a Fibonacci sphere, a Goldberg polyhedron of hexagons and exactly twelve pentagons, with Euler characteristic $V - E + F = 2$ and every edge shared by two faces — carries the AVM in three dimensions. `shape_energy_3d` uses the 3D geometry (a Newell area vector per face, 3D edge lengths)

plus a *lumen-volume* term $K_V(V - V_0)^2$ that keeps the shell inflated against surface tension — the closure a 2D patch got from pinning. The force is one 3D autograd pass; the bounded Euler step carries over unchanged.

One 3D-specific lesson earned its keep. A free surface with area + tension energy and *no bending rigidity buckles* out of plane (roughness $0.004 \rightarrow 0.155$) — exactly as a real epithelium does; the lumen term fixes the total volume but not local smoothness. Adding a soft radial term $K_R \sum_v (|\mathbf{r}_v| - R_0)^2$ (a bending-like penalty toward the shell) recovers a smooth relaxed vesicle: the roughness stays ~ 0.02 and the cross-section stays circular (Fig. 8). This closed, force-balanced shell is the substrate for 3D morphogen-driven budding — the 3D IH/HI reconnections (the 3D half of A-RNR) and a mechanical monolayer (apical/basal, with `cell.volume` elasticity) are the remaining 3D increments.

The vesicle is rendered as a *monolayer*: each cell is drawn as a prism — an apical face (outer), a basal face (inner, the surface scaled radially inward), and the lateral walls between — so a cell carries many faces, not one flat polygon. The apical faces share the mesh vertices and tile edge-to-edge into a closed opaque shell; cells take the Turing white $\rightarrow$ red activation LUT (here activation = 0, all white, awaiting the Goal 2 reaction–diffusion). The cross-section slices the monolayer: each junction edge crossing the plane gives an apical point and its inward-scaled basal counterpart, and the connecting segment is that junction’s lateral wall, partitioning the shell into the individual cells around the lumen (Fig. 8, bottom).

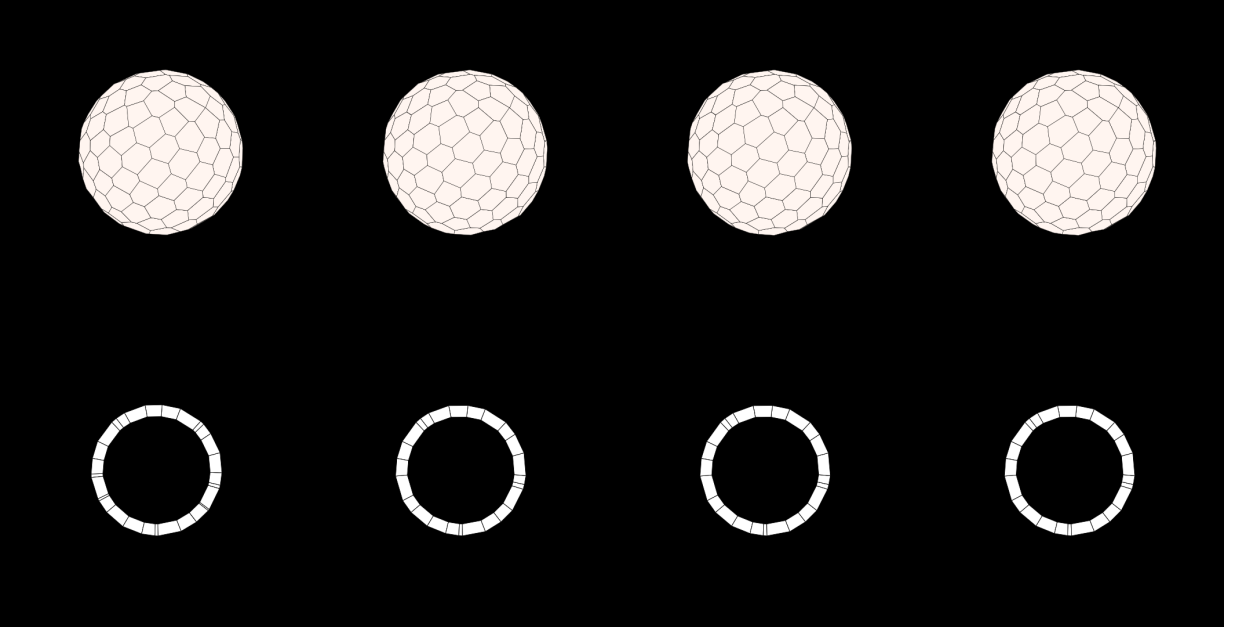


Figure 8: **A 3D epithelial vesicle** (`vesicle_3d`), 0/33/66/100%. *Top*: the 3D monolayer shell — cells as opaque, edge-to-edge prisms (apical/basal/lateral faces), white (activation = 0) with black junctions. *Bottom*: the monolayer cross-section, rendered to the same scale — a ring of individual cells (apical outer, basal inner, lateral walls) around the hollow lumen. A jittered sphere relaxes to a uniform, smooth epithelial shell, held inflated by the volume term and kept from buckling by the soft radial term. The true-vertex-model sibling of the Turing-vertex spherical-Voronoi vesicle.

Growth by per-cell volume elasticity (the emergent inflation)

Growing the vesicle turned out to be the sharpest model-deciphering lesson of the 3D stage. The naive recipe — ramp each cell’s target area A_0 and a single *global* lumen volume V_0 , then let the shape energy inflate the shell — *fails*: the sphere lags its targets and the surplus cell area buckles out of plane into deep crumples (roughness $0.01 \rightarrow 0.2$, shape index climbing from 3.8 to 4.4 — “too much fabric on a ball”). Neither slowing the growth, adding relaxation, nor strengthening a global lumen pump fixed it, because a *single scalar* $K_V(V - V_0)^2$ constrains only the *sum* of the cell volumes: any set of local dips that cancel in the sum is energetically free, so buckling is unopposed.

The fix is the mechanism both references already use. `tyssue`’s `ClosedMonolayer` gives every *cell* a `preferred_vol` and `vol_elasticity` (not just a lumen), and Turing_vertex’s Eq. 3 writes the cell energy as $U = \sum_j [\frac{1}{2}k_v(v_j - v_{eq,j})^2 + \kappa_s s_j]$ — a *per-cell* volume elastic energy. So we split the one global lumen term into a sum over cells: the lumen volume $V = \sum_f v_f$ with per-cell wedge volumes $v_f = \frac{1}{3}(\mathbf{c}_f \cdot \mathbf{N}_f)$ (the pyramid from the sphere centre to face f), and the energy carries $\sum_f K_V(v_f - v_{eq,f})^2$. Now each cell holds *its own* volume: a local dip lowers that cell’s v_f and is penalised, so the term *actively resists buckling*, and growth is just ramping each $v_{eq,f}$ (with $A_0 \propto s^2$, $v_{eq} \propto s^3$). The whole shell then inflates purely by force balance — no vertex is moved by hand — from radius ~ 5 to ~ 7.1 while staying smooth (roughness ≤ 0.01) and keeping the cells at their target shape (index $3.8 \rightarrow 3.87$), Fig. 9. The lesson: *distributed* elastic targets, not a global constraint, are what make morphogenetic expansion emerge and stay stable — the same reason real epithelia use cell-autonomous volume control.

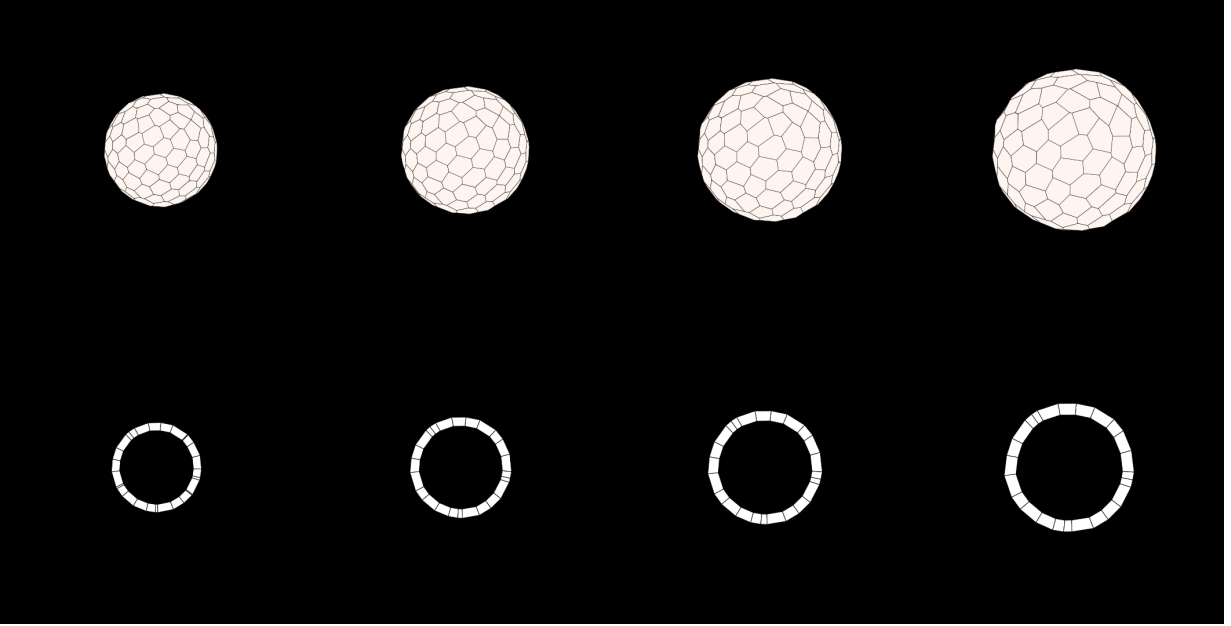


Figure 9: **A growing vesicle** (`vesicle_grow`), 0/33/66/100%, framed to the final extent. Ramping each cell’s target volume v_{eq} inflates the shell through the per-cell volume elasticity alone: the sphere (top) and its cross-section ring (bottom) grow self-similarly from $R \approx 5$ to ≈ 7.1 , the cells stay uniform and well-tiled, and the shell stays smooth (roughness ≤ 0.01). Replacing this per-cell term with a single global lumen volume instead crumples the shell — the emergence of stable expansion requires distributed, not global, volume targets.

Division in 3D (a proliferating vesicle)

The last 3D increment is cell *division* on the closed shell — the sphere analog of tyssue’s `sheet_topology.cell_divi` (a face split by a line; the bulk `monolayer_topology` version with a `vertical/horizontal` orientation waits for the mechanical monolayer). The cell cycle is *volume doubling*: each cell records its birth wedge volume and divides once its current volume reaches twice that (`divide_3d`). A division splits the cell by an *edge-midpoint septum* — two of its edges are cut at their midpoints (m_1, m_2) and a new edge m_1-m_2 separates the daughters. The subtlety unique to 3D: every edge of a closed surface is shared by two faces, so cutting an edge *also* inserts the midpoint into the neighbouring cell’s ring. Getting that bookkeeping right keeps the mesh a valid closed surface: each division adds exactly $\Delta V=2$, $\Delta E=3$, $\Delta F=1$, so $V-E+F$ is invariant. A standalone check (reorienting the SphericalVoronoi outward first, so every directed edge is unique) confirms Euler = 2 across dozens of divisions; *in the engine* the growing, dividing vesicle proliferates $150 \rightarrow 521$ cells over 371 divisions in 220 frames and still verifies `closed = true`, `Euler = 2` (Fig. 10). Pacing follows the Turing-vertex `fig4_vesicle` reference (volume-doubling, a few divisions per frame) so the tissue proliferates gradually. Two subtleties were worth fixing: divisions must be chosen *unbiased* — selecting the first ready cells in face-index order sweeps pole-to-pole because the Fibonacci seeding numbers faces that way, so we shuffle the candidates and randomise the septum direction — and daughters must be kept *compact*. On the last point we followed a literature sweep (tyssue’s `monolayer_topology`, Turing-vertex, cellGPU): the emergent fix is not a smoothing pass but two mechanisms already in the physics. (i) *Long-axis* (Hertwig) division — the septum cuts perpendicular to the cell’s longest axis (an SVD of the cell’s vertices), which yields compact daughters instead of elongated slivers. (ii) *Cortical contractility* $\frac{1}{2}\Gamma \sum_f P_f^2$, the perimeter term that rounds each cell. Together they drop the shape index from ~ 4.2 to ~ 3.9 *emergently* (a geometric Lloyd smoothing would also round the cells, but it is a fairing hack, not physics, so we discarded it). The last step toward the regular-hexagon floor is *T1 neighbour exchange*, the annealing mechanism, now added as `reconnect_t1_3d` (a **Rewire** op, the 3D sibling of the 2D `t1_transition`): every interior edge shorter than a threshold is flipped by a local four-cell neighbour exchange ($A-B$ lose the edge, the far cells $C-D$ gain it), collapsing the edge to its midpoint and reopening it perpendicular in the tangent plane. It keeps V, E, F (and Euler) fixed and only reconnects; a standalone check confirms the shell stays closed with Euler= 2 through 200+ flips. Adding it to the dividing vesicle anneals the division defects and drops the shape index a further $3.96 \rightarrow 3.88$. Note the sphere mandates ≥ 12 pentagonal defects (Euler), so perfect hexagons are not the target. *At scale* (the `fig4` test: $600 \rightarrow 2367$ cells over 1000 frames) the shell stays `closed`, `Euler = 2`, roughness 0.008, shape index 3.84 — rounder at scale, as more cells anneal better.

10 Goal 2: Turing reaction–diffusion on the cell set

The two-level hierarchy pays off again for morphogenesis. The morphogen lives on the *cell* set as state `chem = [a, h]` (a first-class biological DOF, not derivable from vertex geometry), while the mechanical *vertex* set holds the shell. Three operators run the reaction–diffusion each frame: `cell_geometry_3d` (an **Aggregate**: vertices $\rightarrow$ per-cell centroid), `cell_adjacency` (a **Rewire**: the cell–cell neighbour graph) and `cell_diffuse + cell_react` (**Lateral** field updates that integrate `chem`).

Two points are worth stating. First, *how we get cell neighbours without a Voronoi*: in the AVM the cells *are* the faces of the half-edge mesh, so two cells are neighbours iff they share a mesh *edge* — `cell_adjacency` groups the half-edges by their vertex pair and links the two faces that

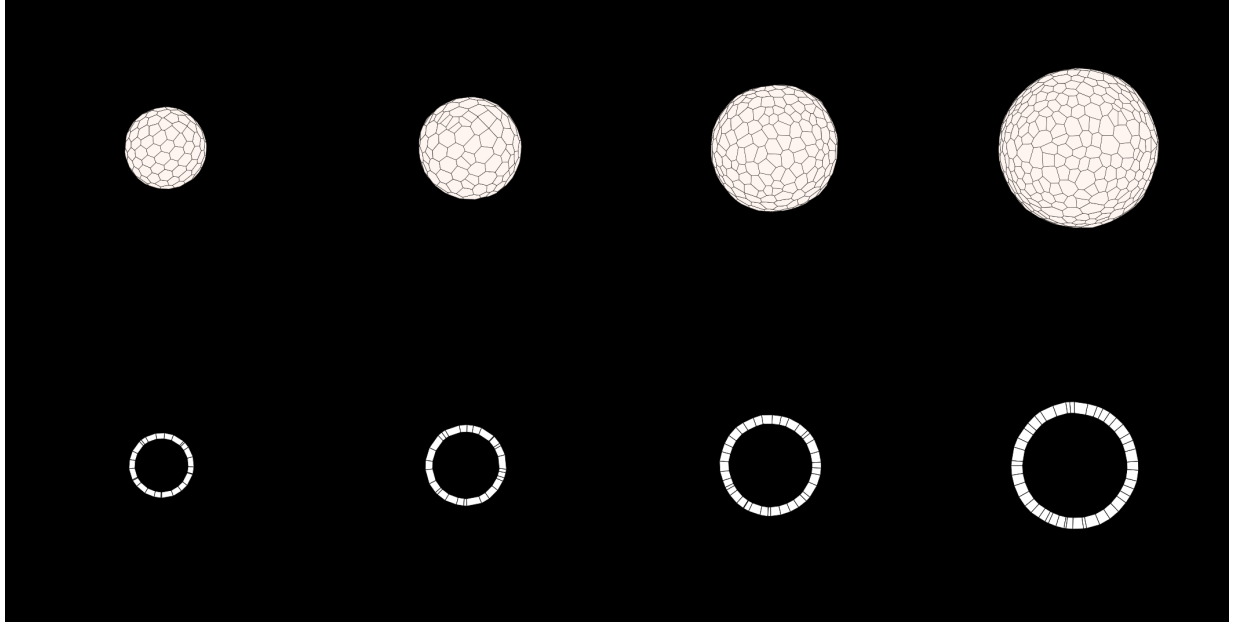


Figure 10: **A dividing (proliferating) vesicle** (`vesicle_divide`), 0/33/66/100%, framed to the final extent. The vesicle grows by per-cell volume elasticity *and* its cells divide by a volume-doubling cell cycle (edge-midpoint septum): the shell inflates from $R \approx 5$ to ≈ 9.5 while the cell count grows $150 \rightarrow 521$, so cells stay near constant size (top: 3D shell; bottom: cross-section, a ring of ever more, smaller cells around the lumen). Division is chosen unbiased (shuffled candidates, random septum direction) so it fills the sphere uniformly. The closed surface stays valid throughout (`Euler` = 2 after all 371 divisions), line tension keeping the fresh daughters well-shaped.

share each edge. The mesh topology already *is* the adjacency graph; no Delaunay/Voronoi of cell centres is needed (that is only how the Self-Propelled-Voronoi sibling builds *its* graph). Second, `react` is a plexus2 *contract* with interchangeable implementations: on the same cell set, `gray_scott` gives a coral/labyrinth and `brusselator` (its parameters transposed verbatim from Turing.vertex `fig4.coral`: $\gamma = 2, A = 1, B = 3; d_a = 0.05, d_h = 0.7$) gives round Turing spots. The Brusselator's fast-inhibitor diffusion is only stable at the `fig4` timestep $dt = 0.02$, so the whole sim runs at that step and `shape_energy_3d` is given the same dt so the mechanics still relaxes fully each frame. A self-organised activator pattern emerges across the closed shell (Fig. 12), coloured white $\rightarrow$ red.

Crucially the pattern couples to *topological change*. By construction `rd_coral_grow` is *exactly the dividing vesicle of Fig. 10 plus the coral colouring*: its mechanical schedule (growth $\rightarrow$ shape energy $\rightarrow$ T1 $\rightarrow$ division) is that of `vesicle_divide` verbatim, and the reaction-diffusion ops only *read* the mesh (aggregate vertices to cell centroids, integrate `chem`) — they never move a vertex. We verified that the hard way: a controlled triple at identical mechanics — `grow_only`, `grow_divide`, and `grow_divide+RD` — gave *byte-identical vertex trajectories* for the last two, so the morphogen is provably pure colouring. The one genuine coupling runs the other way: `divide_3d` propagates the mother's `chem` to its daughters (the kept face retains the mother's state, the new face inherits a copy), so the Turing field rides the proliferating tissue coherently instead of being scrambled by division. At the `vesicle_divide` scale ($150 \rightarrow 518$ cells) the labyrinth develops cleanly on the growing, dividing shell.

The synchronised division wave, and the stochastic cell cycle that breaks it. Scaling the same recipe up — to 500 cells over 1000 frames, for a finer pattern — exposed a real instability, and it took the controlled experiment above to show it was *not* the reaction–diffusion. Because every cell is born at the same volume and grows uniformly, they all cross the volume-doubling threshold *in lockstep*: the cell count climbs in flat +100-per-frame steps (a wave capped only by `max_div`), so the shell is driven under sustained *max-rate* division for the entire growth ramp. Fresh, unrelaxed daughters accumulate faster than the bounded Euler relaxation can smooth them and the (face-referenced) shell roughness at mid-ramp blows up to 0.38 (worst frame 1.7) — the transient “grey” of exposed cell walls — annealing back only once growth plateaus. The fix is the one real epithelia use: *cell-cycle-time variability*. Two knobs make the division *stochastic*: a *stochastic volume seed* (`vseed_cv` on `seed_mesh_3d`) gives each cell a random initial cell-cycle phase so the *first* wave is desynchronised, and a *stochastic cell cycle* (`cycle_cv` on `divide_3d`) draws each daughter’s next division threshold from a Gaussian so later generations stay spread out. With $CV = 0.4$ the lockstep wave becomes a smooth ramp (cells climb gradually from frame 0), the mid-ramp roughness drops two orders of magnitude ($0.38 \rightarrow 0.004$), and `rd_coral_grow_big` proliferates $500 \rightarrow 1994$ cells while staying smooth throughout — a finer coral labyrinth on the larger shell (Fig. 11). The lesson repeats the document’s refrain: the artefact *looked* like a shading or reaction–diffusion bug, but it was a *synchronisation* property of the mechanics, cured by restoring the biological noise a deterministic cell cycle had omitted.

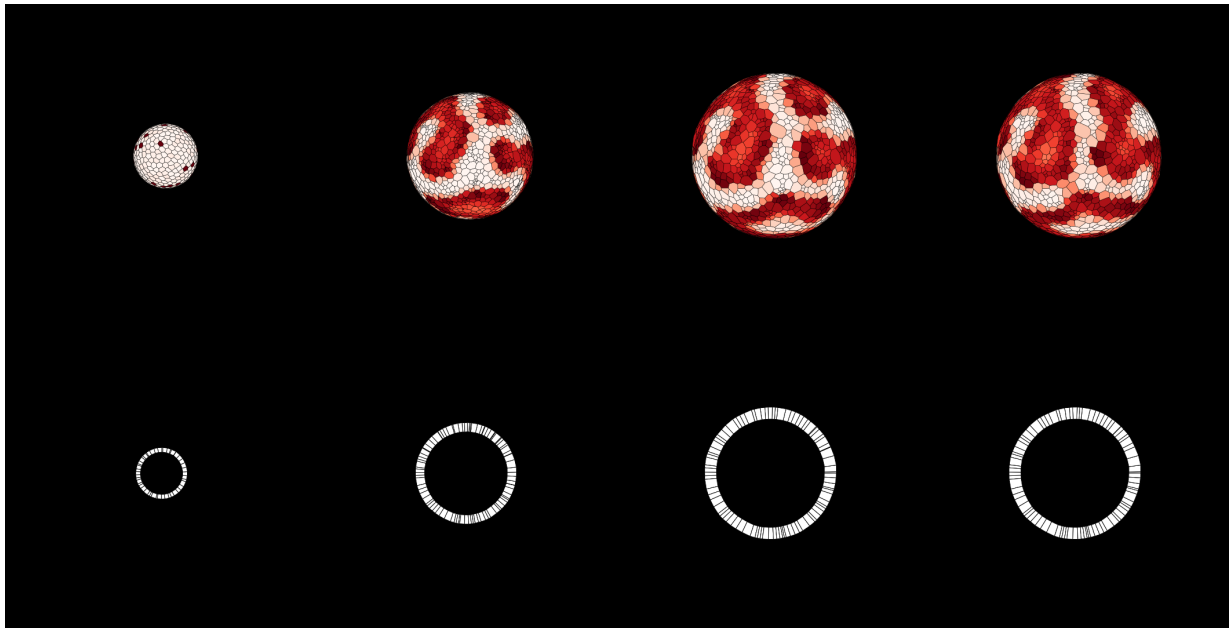


Figure 11: **Coral reaction–diffusion riding a growing, dividing vesicle** (`rd_coral_grow_big`), 0/33/66/100%. The Gray-Scott labyrinth (cell state `chem`, coloured white→red by activator) develops on a shell that grows by per-cell volume elasticity and proliferates $500 \rightarrow 1994$ cells; `divide_3d` hands each daughter a copy of the mother’s morphogen so the pattern stays coherent across division. A *stochastic cell cycle* (per-cell random cycle length, $CV = 0.4$) desynchronises division: without it the cells double in lockstep and the shell roughens $\sim 100\times$ at mid-growth (a grey band of exposed walls); with it the shell stays smooth throughout (top: 3D; bottom: cross-section).

This is the substrate for the morphogenesis proper — morphogen-driven growth (bulge/divide where a is high) on a *fluid* ($p_0 > 3.81$) shell where T1 lets the tissue flow, the bridge to tubes.

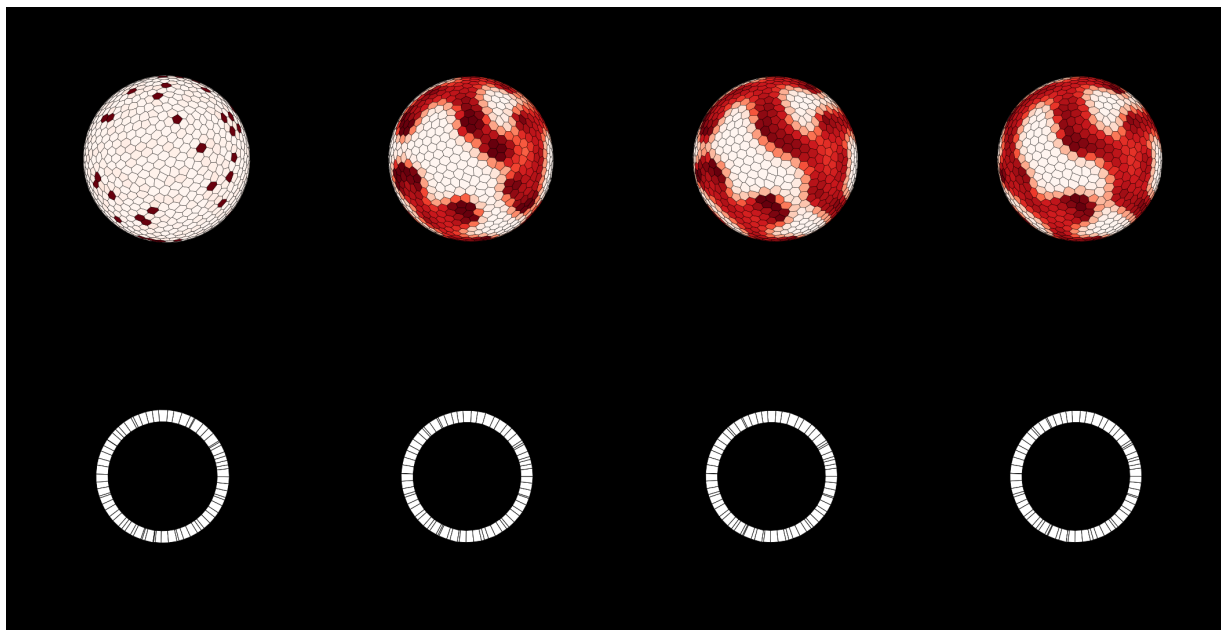


Figure 12: **Live Turing reaction–diffusion on the vesicle cell set** (coral, Gray-Scott), 0/33/66/100%. The morphogen is CELL state; diffusion runs on the cell–cell adjacency read straight from the half-edge table (no Voronoi). From faint noise a self-organised coral/labyrinth activator pattern grows across the closed shell (top: 3D, coloured white→red by activator; bottom: cross-section). The **spots** preset swaps in the Brusselator (fig4 params) on the same set for round Turing spots — **react** is an interchangeable contract.

Field notes

Dated log of what actually worked or failed — the hard-won part.

3D division at scale (2026-07-21) — a synchronised division wave, not a rendering bug.

- **The controlled experiment that exonerated the reaction–diffusion.** `rd_coral_grow` rendered a transient grey band mid-growth that looked like a broken coral or a shading bug. The decisive test was to strip it to a triple at *identical* mechanics — `grow_only`, `grow_divide`, and `grow_divide+RD` — and compare *vertex trajectories*. The last two were byte-identical, so the reaction–diffusion is provably pure colouring; `grow_only` was smooth, so the roughness came from *division under growth*, at scale. A per-frame trace nailed it: the cell count climbs in flat +100/frame steps (all cells doubling together, a wave capped by `max_div`) and the face-referenced roughness spikes to 0.38 (worst 1.7) across the whole ramp. What the eye reported as “grey” was a *synchronisation*, not a mesh error.
- **Stochastic cell cycle: the biological noise the model had omitted.** A deterministic volume-doubling cycle with identical birth volumes *must* synchronise under uniform growth; real tissues don’t, because cell-cycle times vary. Adding that variability — a random initial phase at seed (`vseed_cv`) and a Gaussian per-division threshold (`cycle_cv`) — turns the lockstep wave into a smooth ramp and drops the mid-ramp roughness $0.38 \rightarrow 0.004$, letting the coral scale to $500 \rightarrow 1994$ cells cleanly. One parameter — but only after the controlled experiment localised the cause; guessing at shading or T1 tuning would have chased the

symptom.

- **Honest metrics matter (again).** The first roughness number was inflated by unused reservoir vertices sitting at the origin. Restricting the metric to *face-referenced* vertices — those that actually tile the shell — gave the true $0.38 \rightarrow 0.004$ and matched the rendered strip. Same lesson as the bow-tie faces below: measure the geometric thing you mean.

Stage 2 (2026-07-21) — topology operators, and where the cheap division breaks.

- **Revert-on-invalid is the right contract for topology.** Rather than reason about every degenerate case up front, apply the flip/split, validate the touched faces, and roll back if any is degenerate. T1 then *never* tangles the mesh — a refused flip is just a no-op. This turned an error-prone operation into a safe one and is why the self-test is clean.
- **Through-vertex division is a 1–2 round operator, not a proliferation engine.** Splitting a face by a septum between two of its *existing* vertices adds no new vertices, so each split roughly halves a cell using the same corners; after a couple of rounds cells become triangles and further splits are refused. Worse, under *uniform growth in a pinned patch* the target area outruns the available space, every cell is driven below target, and the sliver daughters collapse — a run with 231 forced divisions ended as a red tangle (79–157 invalid faces). The clean result above deliberately does *one* desynchronised round with no global growth. The proper fix (tyssue’s own) is *edge-midpoint* division: bisect two opposite edges, insert new vertices (shared with the neighbours across those edges), and run the septum between them — well-shaped daughters, sustainable for many rounds. That needs a vertex buffer with occupancy and neighbour-edge bookkeeping; it is the next increment.
- **T1-mediated fluidisation (Stage 3) — first not sound, then fixed by a *bounded Euler* step.** The naive attempt failed its own bar: gentle self-propulsion fired 0 T1s, and the drive needed to collapse edges made the floppy ($p_0=4.1$) sheet *diverge* under a fixed-step relaxation (positions overflow, ~ 85 invalid faces) *before* a T1 could fluidise it — the instability was in the *integrator*, not the reconnection (near the transition the shape energy is nearly flat, so an explicit step lets vertices cross). The sound fix is a *bounded overdamped Euler* step: cap every per-substep move at 15% of the mean edge length with `torch.clamp` — so no vertex can cross a cell in one step, and the step stays *differentiable* (no `scipy` solver; the rollout must remain differentiable for inverse modelling). With that, the sheet stays stable and T1s fire.
- **A second, subtler bug the naked eye caught: bow-tie faces (a false “valid”).** The first “sound” dichotomy ($23\times$) was *wrong*. The rendered strips showed black star-shaped gaps where cells overlapped: the T1 flip was producing *self-intersecting* (bow-tie) faces, and my validity guard — positive shoelace area, no repeated vertices — *passed* them, because a bow-tie has positive shoelace area. So “0 invalid faces” was a false pass and the big T1 counts were mostly bad flips. Two fixes: (i) a proper *simple-polygon* test (no two non-adjacent edges cross) plus a *non-overlap* test on the four faces a flip touches, added to the accept criterion; and (ii) the flip now *searches* the insertion side (which side of its anchor the gained vertex goes) $\times$ the mirror sign, and commits only a genuinely simple, non-overlapping, edge-sharing result. With the honest check the corrected dichotomy is:

v_0	l_{th}	solid T1	fluid T1	ratio
0.06	0.25	96	1413	14.7×

The fluid still performs $\sim 15\times$ the T1s of the caged solid (fluid MSD 0.64 vs solid 0.10 — cage-breaking), now with a *genuinely* valid, non-overlapping tiling at every frame (the strips tile cleanly). The lesson is the one the whole document keeps re-learning: a numeric invariant (area > 0) is not the same as the geometric one you meant (a simple, embedded tiling) — and a rendered movie is a sharper validity check than a scalar. The corrected numbers are smaller than the bogus ones, but the physics — fluid flows, solid jams — survives.

Stage 1 (2026-07-21) — force balance is one line, and it just works.

- The inner-relaxation-as-velocity trick (§3) reaches force balance with no stability tuning: Δt is a free label, `eta`= 0.08, 20 inner steps/frame. Contrast the SPV prototype, whose entire “noisy pattern” saga was a *timestep* artefact. Moving the DOF from cell centres to mesh vertices *and* minimising to residual removed that whole failure mode.
- **The transition sits where the topology puts it.** With a fixed honeycomb (mostly hexagons) the residual energy already collapses across $p_0 \approx 3.72$ –3.81: the single-cell floor for a regular hexagon is $p_0^*(\text{hex}) = 3.722$, so $\langle q \rangle$ cannot fall below ≈ 3.72 no matter how small p_0 is (the **solid** cases pin there). The canonical 3.81 is the *disordered*-polygon value; sharpening the transition to exactly 3.81 needs T1 rearrangements to populate the pentagon/heptagon classes — the next operator. So the transition is not a fixed number but a readout of the polygon-class distribution, which is a topology property.
- **The engine calls forward() under no.grad.** The autograd force needs an explicit `with torch.enable_grad()` (as the SPV op also did) — otherwise `torch.autograd.grad` raises “does not require grad”. Cost one run to rediscover.
- **Pin the border or the patch curls.** A free (unpinned) bounded sheet contracts under perimeter tension and the boundary ring buckles inward, contaminating the bulk statistics. Pinning vertices of degree < 3 (those on the ragged Voronoi border) holds the frame; bulk stats exclude any face touching a pinned vertex.

[next] morphogen-driven tubes and branching. The structural increments that were “next” in the earlier notes are now *done*: the genuine two-level hierarchy (a `cell` set carrying `chem`, fed by a `cell_geometry` Aggregate), edge-midpoint division with a decidable long-axis angle, and the lift to the closed 3D monolayer (per-cell volume elasticity + surface T1) all exist and are exercised above; the reaction–diffusion runs on the cell set and rides division; and the division wave is tamed by a stochastic cell cycle. What remains are the morphogenesis *targets* from Turing_vertex, and they reuse every operator above unchanged — only the coupling and the fluidity are new dials. (i) *Morphogen-driven growth*: `morphogen_growth_3d` grows a cell’s target volume where the activator a is high, so a localised Turing patch on a *fluid* ($p_0 > 3.81$) shell, where T1 lets the tissue flow and its cells divide (handing the activator to their daughters), should *bud and extend into a tube* rather than jam into a spike (Turing_vertex Fig. 5). (ii) Lowering the reaction–diffusion feed drives that patch to *branch* (Fig. 6). The exploratory **tube** runner — the same schedule with the morphogen→growth coupling switched on and p_0 raised into the fluid regime — is the current front; stabilising the bud against the fluid shell’s tendency to balloon is the open problem.